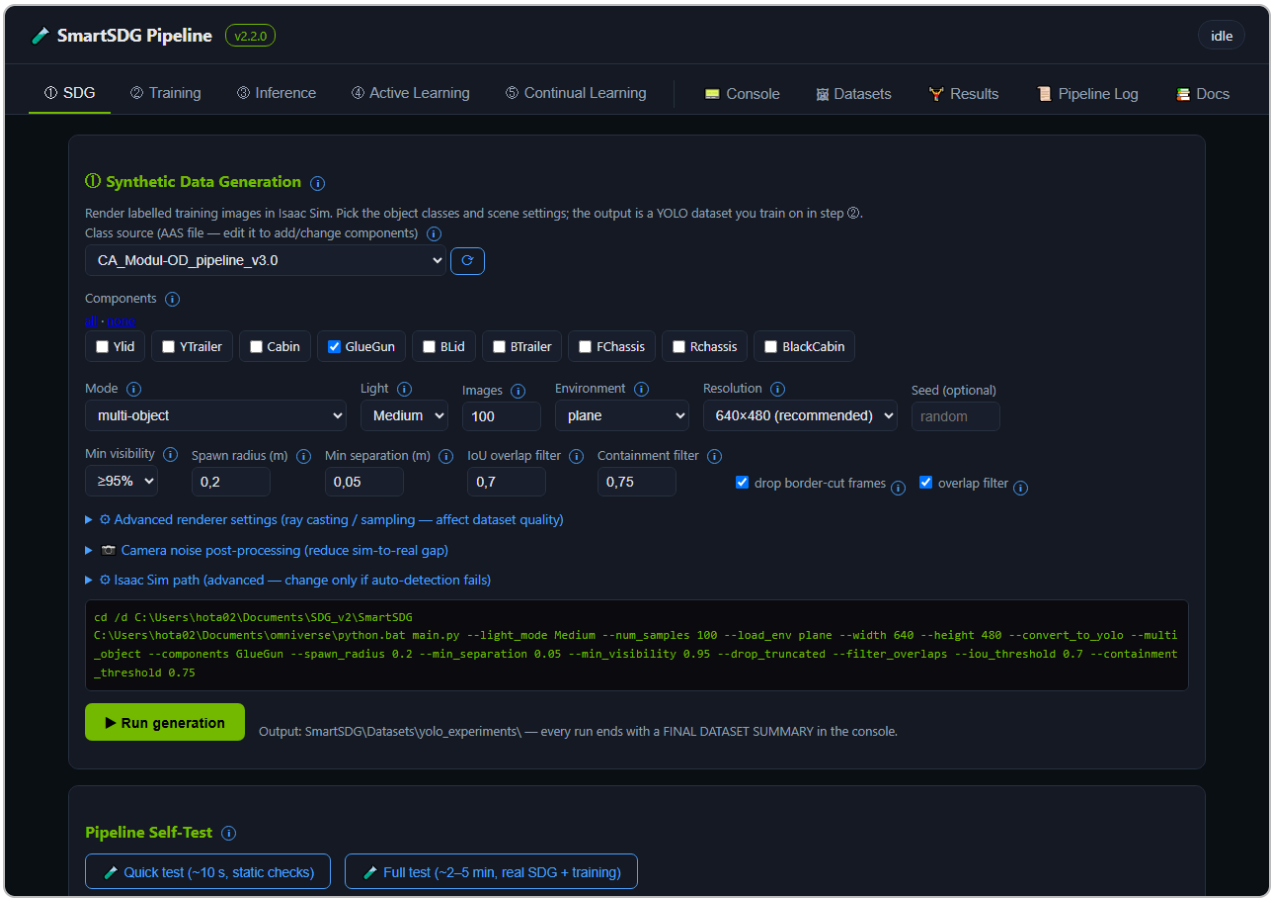
SmartSDG Pipeline

Comprehensive User Guide

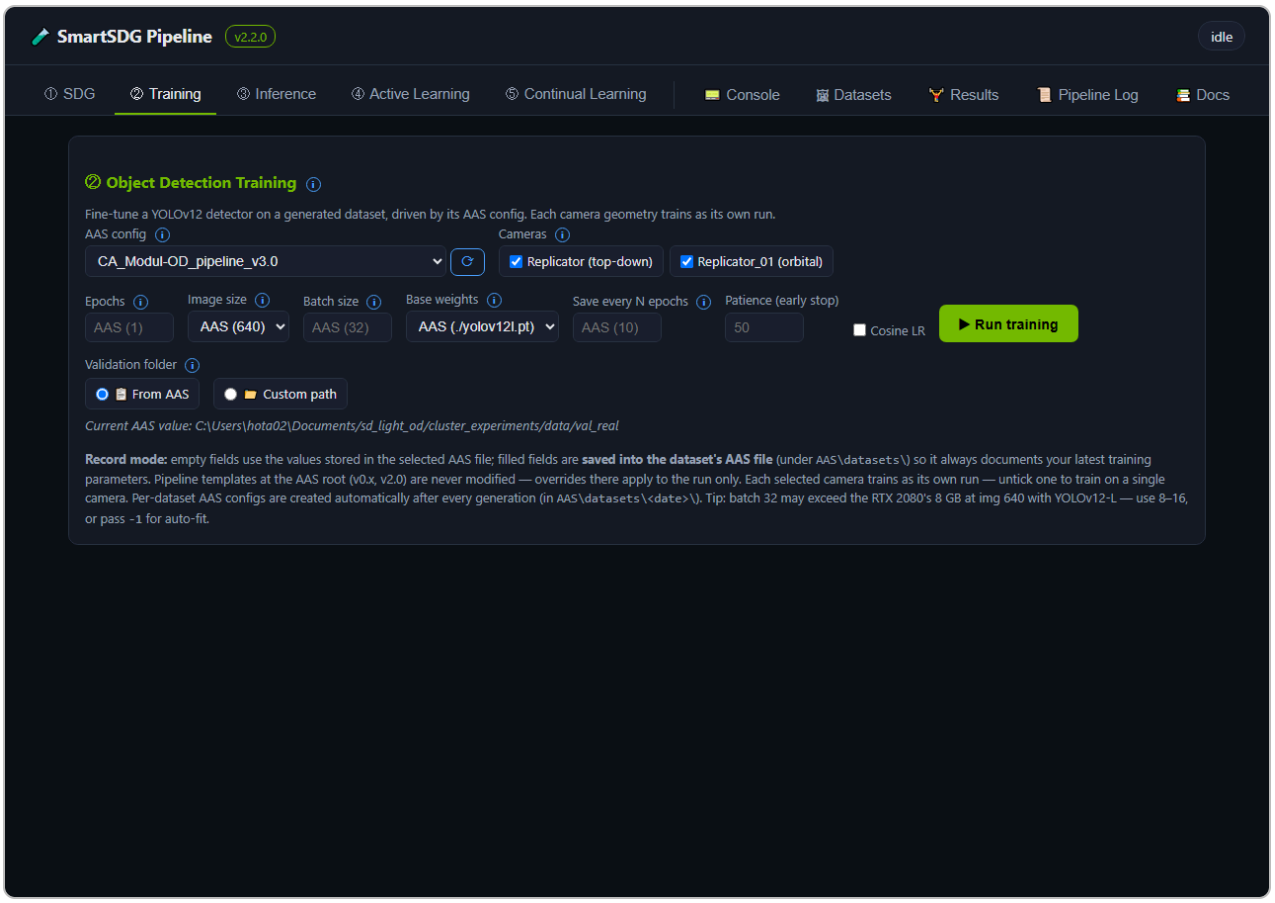
Synthetic Data Generation → Training → Inference → Active Learning

Version 2.2.0 · generated 2026-06-18

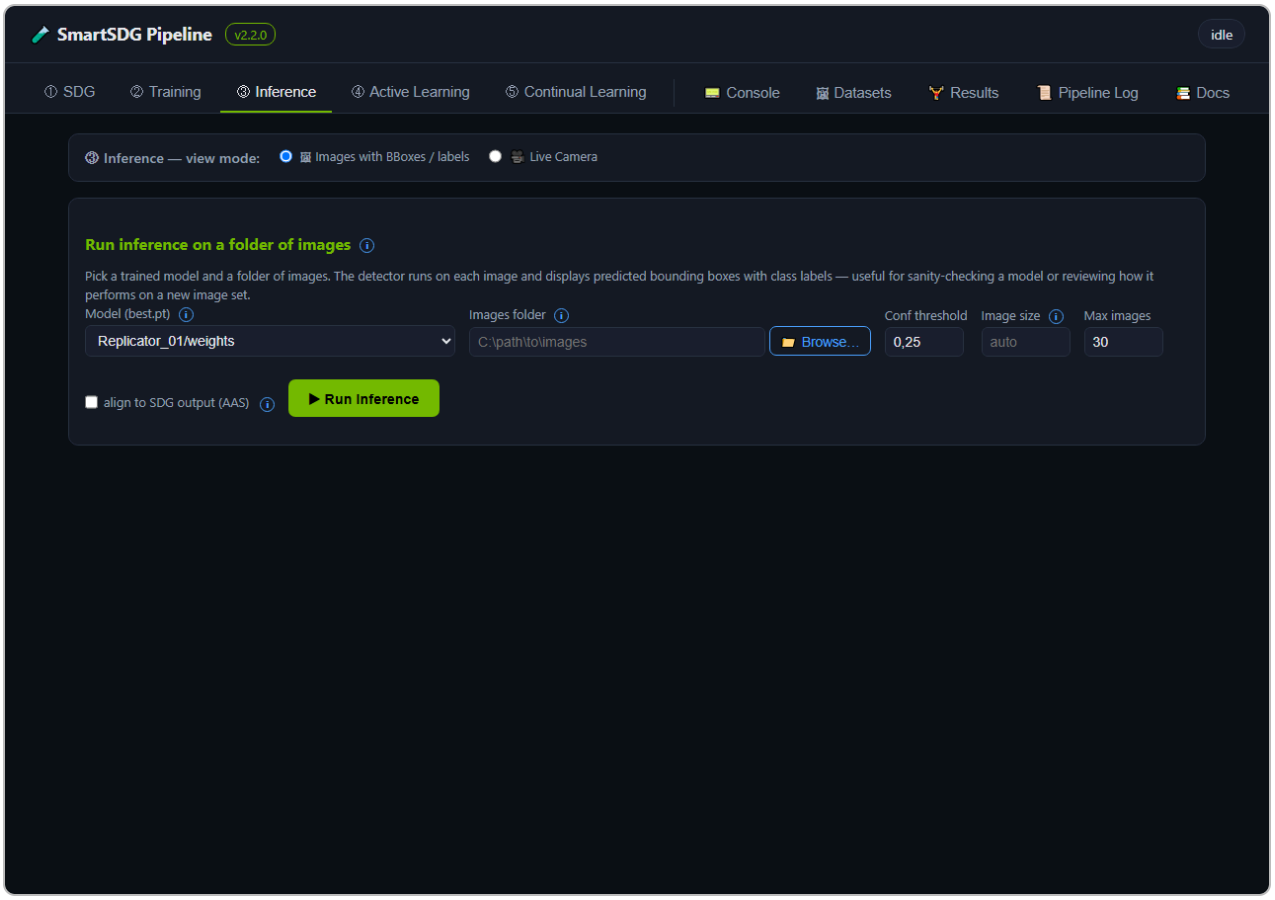
The dashboard at a glance



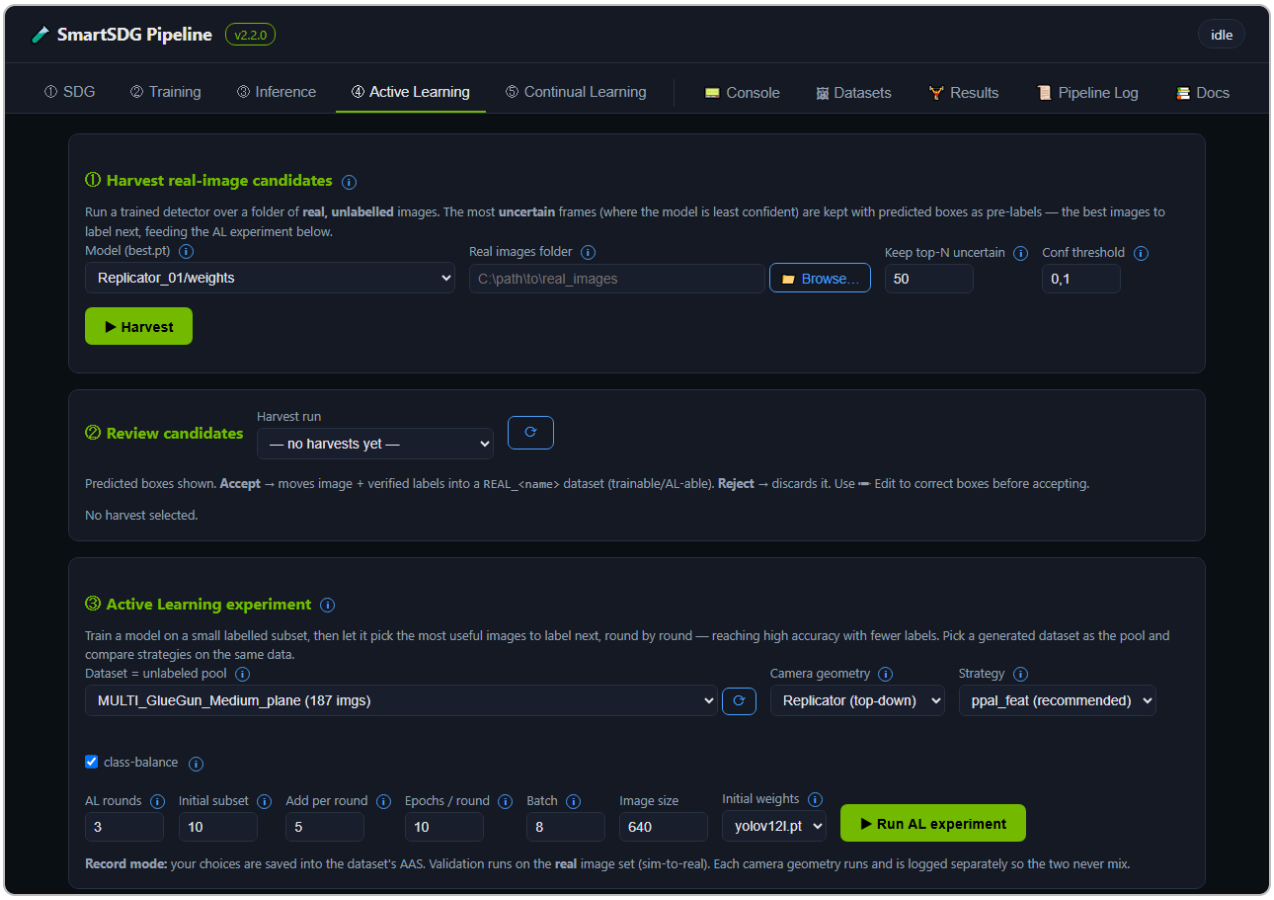
Synthetic Data Generation — pick components, scene and quality filters



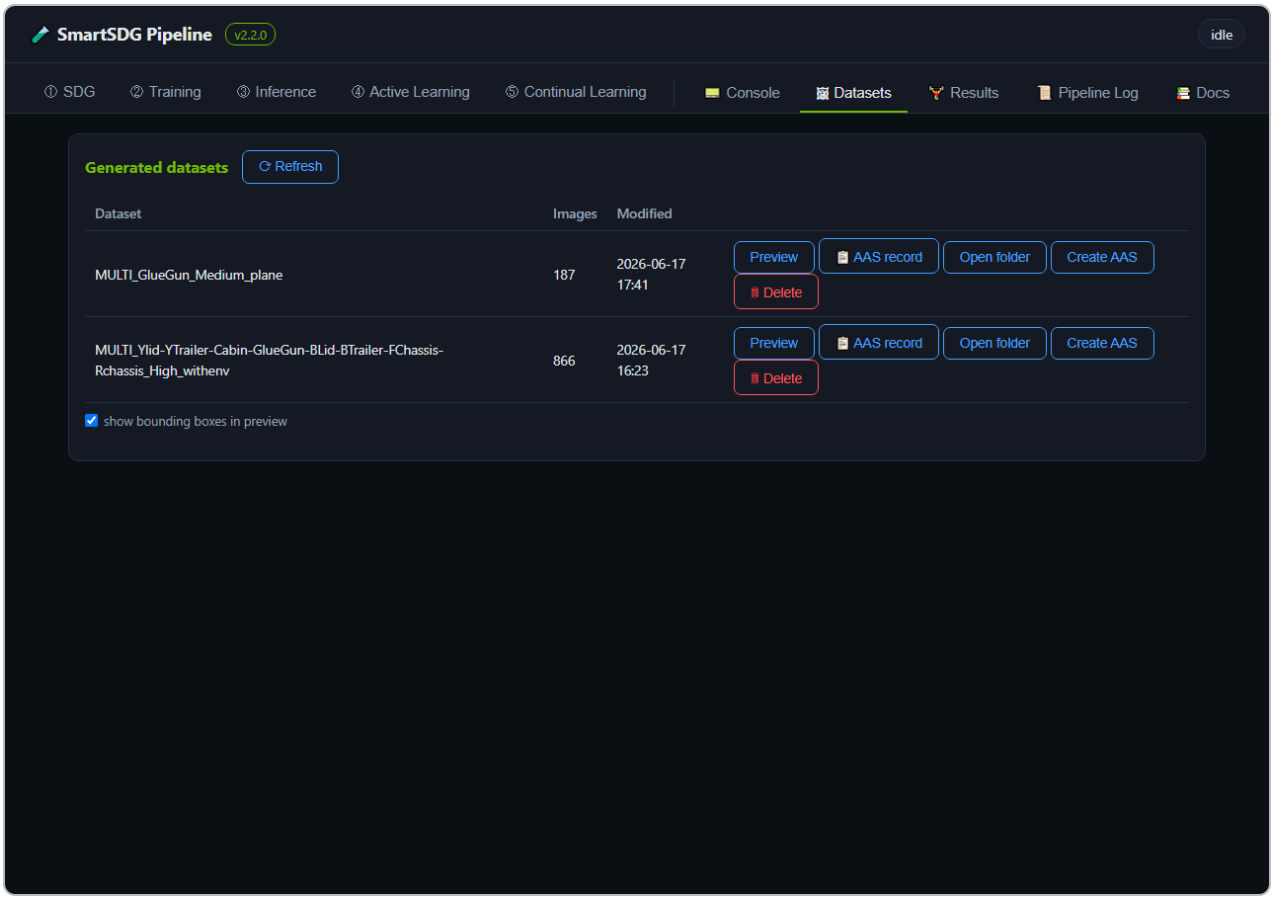
Object Detection Training — per-camera, record mode



Inference — Images mode (BBoxes) or Live Camera (webcam/IP cam/video)



Active Learning — Harvest → Review candidates → AL experiment + gain chart



Datasets — browse, preview boxes, edit labels, delete

SmartSDG Pipeline — Comprehensive User Guide

> A complete, example-driven manual for the whole pipeline, written for a professional user who is **not** expected to be an expert in synthetic data, object detection, or active learning. Every control in the dashboard has an ⓘ button whose "Open full guide" link brings you here.

Table of contents

1. What this pipeline does
2. The big picture (diagram)

3. Starting the dashboard
4. Step ① — Synthetic Data Generation (SDG)
5. Step ② — Object Detection Training
6. Step ③ — Inference & harvesting real candidates
7. Step ④ — Active Learning
8. Labelling & the box editor
9. Where everything is stored
10. Reproducibility & the AAS record
11. Worked end-to-end example
12. Troubleshooting
13. Glossary

---

1. What this pipeline does

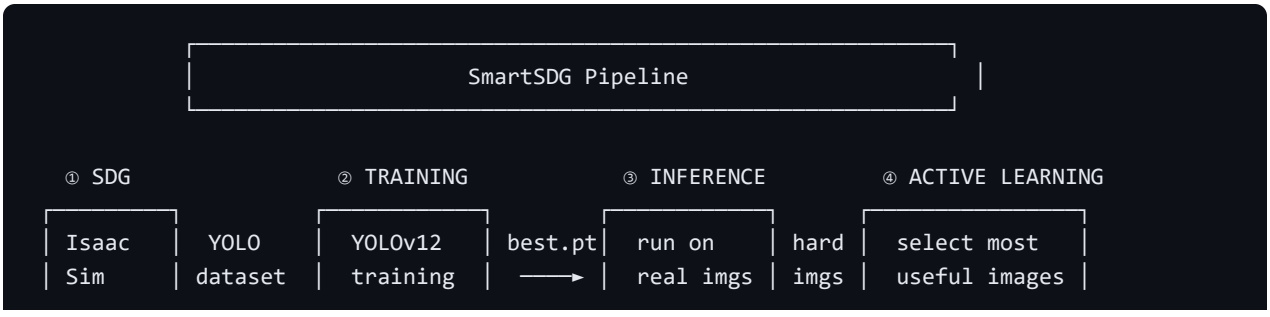
You want an object detector that works on **real** images of industrial truck components — but labelling thousands of real photos by hand is expensive. This pipeline lets you:

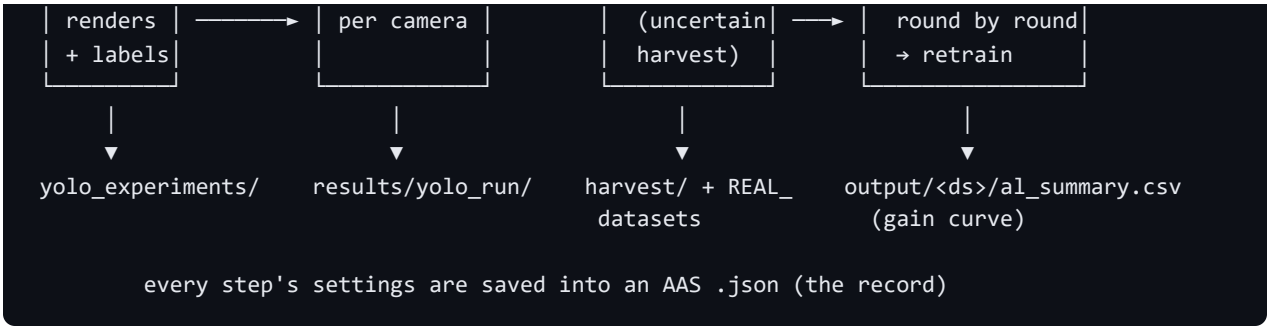
1. **Generate** perfectly-labelled training images in a 3D simulator (no photos, no manual labelling).
2. **Train** a YOLOv12 detector on them.
3. **Run** that detector on real images to find the ones it struggles with.
4. **Actively learn**: label only those most-useful images and retrain, reaching high accuracy with a fraction of the labelling effort.

Everything is driven from one local dashboard and recorded in AAS config files so any experiment is reproducible.

---

2. The big picture





Each scene is rendered by **two cameras** — `Replicator` (fixed top-down) and `Replicator_01` (random orbital) — and these are kept as **separate datasets** throughout, so you can train and compare each viewpoint independently.

---

3. Starting the dashboard

With Docker (recommended — no Python install needed)

```
:: Build the image once (first time only — ~5 GB download):
scripts\build_docker.bat

:: Then start the dashboard:
docker compose -f docker/docker-compose.yml up dashboard
```

Opens **http://localhost:8765**.

Enable Docker mode in `pipeline_config.json` (project root, git-ignored):

```
{
  "docker_enabled": true,
  "val_real": "C:/path/to/your/val_real"
}
```

With Docker mode on, training / AL / CL runs are automatically routed into the container (`--gpus all --shm-size=2g`). SDG and live inference always run natively.

Without Docker (conda env)

```
:: activate the training environment first, then:
conda activate yolov12
scripts\launch_dashboard.bat
```

This opens `http://127.0.0.1:8765`. The top bar shows a **status badge** (idle / running / finished). The tabs follow the pipeline order:

```
① SDG | ② Training | ③ Inference | ④ Active Learning | ⑤ Continual Learning | Console | Dataset
```

The ③ **Inference** tab has a mode picker: **Images** (run model on a folder, view BBoxes) or **Live Camera** (webcam / IP cam / video MJPEG stream). The ④ **Active Learning** tab contains all three phases: Harvest → Review → AL experiment.

Any long-running job streams its output live to the **Console** tab.

---

4. Step ① — Synthetic Data Generation

Goal: produce a labelled YOLO dataset.

Controls

- **Class source** — the AAS file whose class list defines the objects and their global class IDs. To add a new object: add its name here and drop `<Name>.usd` into `SmartSDG/assets/Incorap_lkw/`; it auto-registers with the next free ID.
- **Components** — tick the objects to render. Grayed = missing USD file.
- **Mode** — `multi-object` (several parts per scene, for detection) or `single-object`.
- **Light / Images / Environment / Resolution / Seed** — scene settings. Seed makes a run fully reproducible (same object positions, rotations, lighting).
- **Quality filters** — min-visibility, spawn radius, min-separation, IoU overlap, drop border-cut. These guarantee every kept image contains only fully-visible, fully-labelled, non-overlapping objects (see HYPERPARAMETERS.md).
- **Advanced renderer** — samples-per-pixel, bounces, denoiser (render quality).

Example

Generate 100 multi-object images of three parts at 640×480 on a plane:

```
Components: Cabin, GlueGun, BLid
Mode: multi-object   Light: Medium   Images: 100   Environment: plane
Resolution: 640×480  Min visibility: ≥95%  Spawn 0.2  Separation 0.05
```

Press ▶ **Run generation**. The Console ends with:

```
##              FINAL DATASET SUMMARY                       ##
  Frames rendered by Isaac Sim       : 200
  Frames discarded - object occluded < 95% visible : 31
  FINAL images in dataset            : 142
  Output: ...\yolo_experiments\MULTI_Cabin-GlueGun-BLid_Medium_plane
```

The dataset now appears in the **Datasets** tab (with thumbnails + box preview).

---

5. Step ② — Object Detection Training

Goal: train a YOLOv12 detector on a generated dataset.

- **AAS config** — the dataset's config (auto-created after generation, listed at the top of the dropdown).
- **Cameras** — `Replicator` and/or `Replicator_01`. **Each ticked camera trains as its own separate run** — the two viewpoints never merge.
- **Epochs / Image size / Batch / Base weights / Save period** — standard training knobs. Empty fields use the AAS values; filled fields are saved back into the dataset's AAS (**record mode**).

  Memory tip: on an 8 GB RTX 2080, batch 32 at 640px with YOLOv12-L can run out of memory — use batch 8–16, or `-1` for auto-fit.

Press ▶ **Run training**. Results (weights `best.pt`, PR/F1 curves, confusion matrices) land in `OD_training/results/yolo_run/exp{0,1,2}/<dataset>/` and show up in the **Results** tab, one row per camera run, with a Weights ✓/— column.

---

6. Step ③ — Inference

The ③ **Inference** tab has two modes, selected by the radio button at the top:

Mode A — Images with BBoxes / labels

Run a trained model over a folder of images and view the detections.

1. **Model** — pick a `best.pt` from your training runs.
2. **Images folder** — click 📂 **Browse…** to navigate to the folder.
3. **Conf threshold** — boxes below this confidence are not drawn.
4. **Max images** — how many images to process (default 30).

Press ▶ **Run Inference**. Every image is processed and displayed with predicted bounding boxes and class labels. Use this to quickly sanity-check a model on any image set — no saving, no scoring, just visual results.

Mode B — Live Camera

Stream annotated detections in real time. Three source types:

Source	How
Webcam	Select the camera index (click **detect** to probe 0–3)
IP Camera	Paste the full RTSP or HTTP URL, e.g. `rtsp://admin:pass@192.168.1.10:554/stream`
Video file	Browse to an `.mp4 / .avi / .mov / .mkv` file — plays through once

Press ▶ **Start**; the server opens the source and streams annotated frames to the browser as MJPEG. Press ■ **Stop** to release the camera / stream.

> **Note:** Live webcam does not work inside Docker on Windows (USB passthrough is unreliable). Run the dashboard natively for webcam use. IP cameras on the same network work from both native and Docker modes.

---

7. Step ④ — Active Learning

The ④ **Active Learning** tab has three sections in order:

① Harvest real-image candidates

Goal: use a trained model to find the **real** images most worth labelling.

1. **Model** — pick a `best.pt` from your training runs.
2. **Real images folder** — click 📂 **Browse…** to navigate to the folder.

3. **Keep top-N uncertain** — how many of the hardest frames to keep.
4. **Conf threshold** — detection confidence used while scoring.

Press ▶ **Harvest**. The model scores every real image by uncertainty and keeps the top-N, saving each with its predicted boxes as **pre-labels**.

② Review candidates

After Harvest, select a harvest run from the dropdown. Each candidate is shown with its predicted boxes:

- ✏ **Edit** — open the box editor to correct the labels (see §8).
- ✓ **Accept** — moves the image + labels into a `REAL_<name>` dataset (then trainable / AL-able like any dataset).
- ✕ **Reject** — discards it.

This is the human-in-the-loop: the model tells you which real frames are worth your labelling time.

③ Active Learning experiment

Goal: reach a target accuracy with far fewer labelled images, by letting the model choose what to label next. Full details in ACTIVE_LEARNING.md.

- **Dataset** = the unlabelled pool (a generated dataset).
- **Camera geometry** — run one camera at a time; each is logged separately so the two compare independently.
- **Strategy** — `random` (baseline) / `entropy` / `ppal_box` / `ppal_feat` (recommended).
- **Class-balance** — boosts rare classes.
- **Budget** — initial subset, rounds, add-per-round.

Each round trains a model, scores the pool, and adds the selected images. The **Gain overview** chart plots **mAP50 vs number of labelled images** for every run. To prove AL works, run the same dataset+camera with `random` and with `ppal_feat` and compare the two lines — the gap is the gain.

---

8. Labelling & the box editor

Click ✏️ **Edit** on any harvest candidate or dataset thumbnail to open the editor:

Action	How
New box	drag on an empty area
Select	click a box
Move	drag inside a selected box
Resize	drag the bottom-right corner handle
Delete	Del key, or the delete button
Change class	the class dropdown (applies to the selected/new box)

💾 **Save** writes standard YOLO labels (`class cx cy w h` , normalised). Files are plain text, so you can also refine them in LabelImg / Roboflow / CVAT.

---

9. Where everything is stored

What	Path
Synthetic dataset	`SmartSDG/Datasets/yolo_experiments/<name>/`
Real (accepted) dataset	`SmartSDG/Datasets/yolo_experiments/REAL_<name>/`
Raw render (pre-conversion)	`SmartSDG/Datasets/unified_render_output/<name>/`
Training runs	`OD_training/results/yolo_run/exp{0,1,2}/<name>/`
Harvest candidates	`active_learning/harvest/<name>/{images,labels}`
AL outputs + gain curve	`active_learning/output/<name>[__<camera>]/al_summary.csv`
Config record (per dataset)	`AAS/datasets/<date>/CA_Modul-OD_dataset_<name>.json`

`exp0` = void background, `exp1` = plane, `exp2` = full 3D environment.

---

10. Reproducibility & the AAS record

Every dataset gets an **AAS** (Asset Administration Shell, IEC 63278) JSON file that records how it was generated and trained. The dashboard runs in **record mode**: when you change training or AL parameters, they are written into that file, so the dataset and the exact configuration that produced each result always travel together. These files are committed to git, giving a full history of every parameter change (`git diff`).

For exact run reproduction, set a **seed** in SDG — the scene geometry and labels become byte-identical across runs (rendered pixels match up to GPU floating-point noise, the standard caveat for any GPU renderer).

---

11. Worked end-to-end example

A complete cycle, all from the dashboard:

```
① SDG:        generate 100 images (Cabin, GlueGun, BLid), Medium, plane, 640×480
              → dataset MULTI_Cabin-GlueGun-BLid_Medium_plane (≈142 imgs after filtering)

② Training:   AAS = that dataset, camera = Replicator only, 50 epochs, batch 8,
              weights yolov12l.pt → best.pt in results/.../exp1/...

③ Inference:  [Images mode] model = that best.pt, Browse to your images folder,
              conf 0.25 → Run Inference → view results with BBoxes
              [Live Camera mode] pick webcam or RTSP URL → Start → live stream

④ AL (Harvest):  model = that best.pt, Browse to your real-images folder,
              keep top 50, conf 0.10 → Harvest
              → review the 50 hardest real frames, Edit/Accept the good ones
              → creates dataset REAL_<folder>

④ AL (experiment): dataset = the synthetic pool, camera = Replicator,
              strategy ppal_feat, 3 rounds, init 10, add 10
              → run again with strategy random
              → Gain overview shows ppal_feat above random = AL works
```

---

12. Troubleshooting

Symptom	Fix
`ModuleNotFoundError: aas_core3` (training)	activate the `yolov12` env before launching the dashboard, or `pip install aas-core3.0rc02`

Symptom	Fix
Generation ends with `No valid data folders`	a config/parse error — run **Pipeline Self-Test** (Quick) to pinpoint it
`ParserError ... expected '<document start>'`	a config or AAS file has a UTF-8 BOM; the self-test "no BOM" check flags it
Training "Skipping … Path not found"	the AAS `base_data_dir` / camera names don't match the dataset folders
Out of GPU memory	lower batch size, or use `-1` for auto-fit; reduce image size
Dashboard shows "server offline"	the local server stopped — relaunch `scripts\launch_dashboard.bat`

Run `scripts\run_tests.bat` (or the **Pipeline Self-Test** buttons) after any change — it checks configs, assets, the class-ID registry, converter filters, exp routing, and (full mode) a real generation, training and AL loop.

---

13. Glossary

- **SDG** — Synthetic Data Generation (rendering labelled images in a simulator).
- **YOLO label** — a `.txt` per image; each line `class cx cy w h` (normalised).
- **Class ID** — the integer for each component, fixed by the registry order in `config.yaml`; never reordered so old datasets stay valid.
- **AAS** — Asset Administration Shell; the JSON config/record per dataset.
- **Camera geometry** — Replicator (top-down) vs Replicator_01 (orbital); kept separate.
- **Active Learning (AL)** — iteratively labelling the most informative images.
- **mAP50 / mAP50-95** — mean Average Precision, the standard detection accuracy metrics.
- **Uncertainty / diversity** — the two ingredients of AL selection: how unsure the model is, and how different the chosen images are from each other.
- **Record mode** — the dashboard saves your parameters into the dataset's AAS.

Physics-Accurate Illumination in Synthetic Data Generation

Why Environment-Based Global Illumination Outperforms Dome Lights and HDRI for Industrial Object Detection

---

1. The Sim-to-Real Gap and Its Lighting Component

A persistent challenge in synthetic data generation (SDG) for object detection is the **sim-to-real domain gap**: a detector trained exclusively on synthetic images often fails on real images, not because the object geometry is wrong, but because the *appearance* of that geometry under real-world lighting conditions is not captured faithfully in the simulator.

The domain gap has two main components:

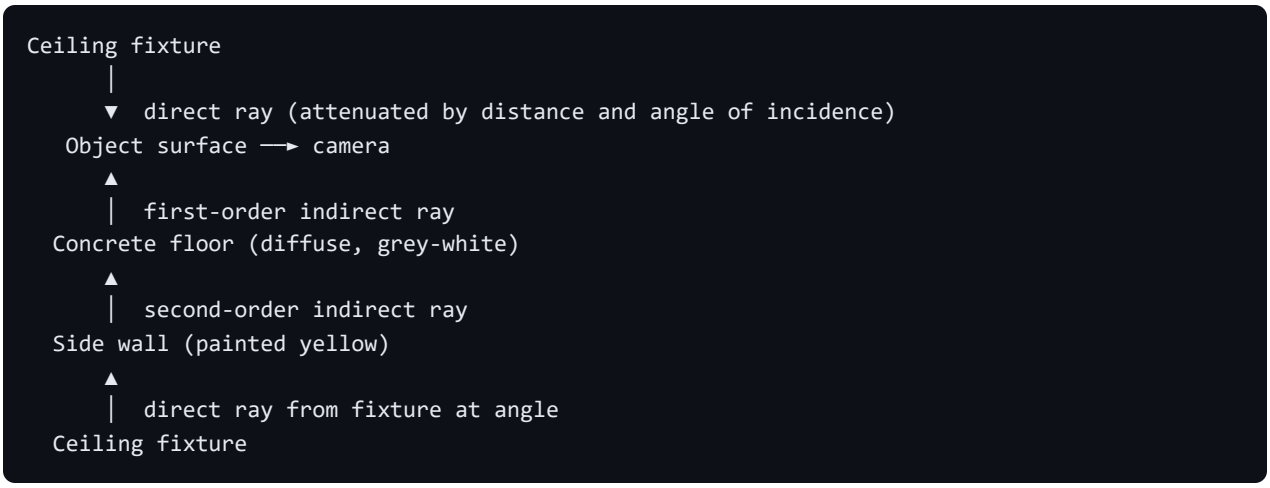
Component	Typical mitigation strategies
Geometric	Accurate 3D models, realistic textures, varied viewpoints
Photometric	Accurate light transport, material BRDFs, sensor noise simulation

Most published SDG pipelines focus heavily on geometric accuracy and treat illumination as a secondary concern — often using simple analytical light models (point lights, directional lights) or image-based lighting (IBL) via dome lights and HDRI maps. This chapter demonstrates why those approaches are insufficient for industrial deployment scenarios and explains the physics-accurate alternative implemented in this pipeline.

---

2. Background: How Light Reaches an Object in the Real World

In a real industrial environment, the appearance of an object on a camera sensor is determined by the complete **light transport history** of every ray that reaches it. A ray from a fluorescent ceiling fixture does not simply travel directly to the object. It may follow a path such as:



The object therefore receives: - **Direct illumination**: rays arriving unobstructed from the source. - **First-order indirect**: rays that bounced once from the floor, walls, or other objects. - **Higher-order indirect**: rays with two or more bounces, carrying colour and intensity information from the entire spatial context of the room.

This multi-path light transport is what gives real photographs their characteristic **soft shadows**, **colour bleeding** (a yellow wall casting a warm tint on a grey component nearby), and **ambient fill** that varies with the geometry and surface reflectance of every object in the room.

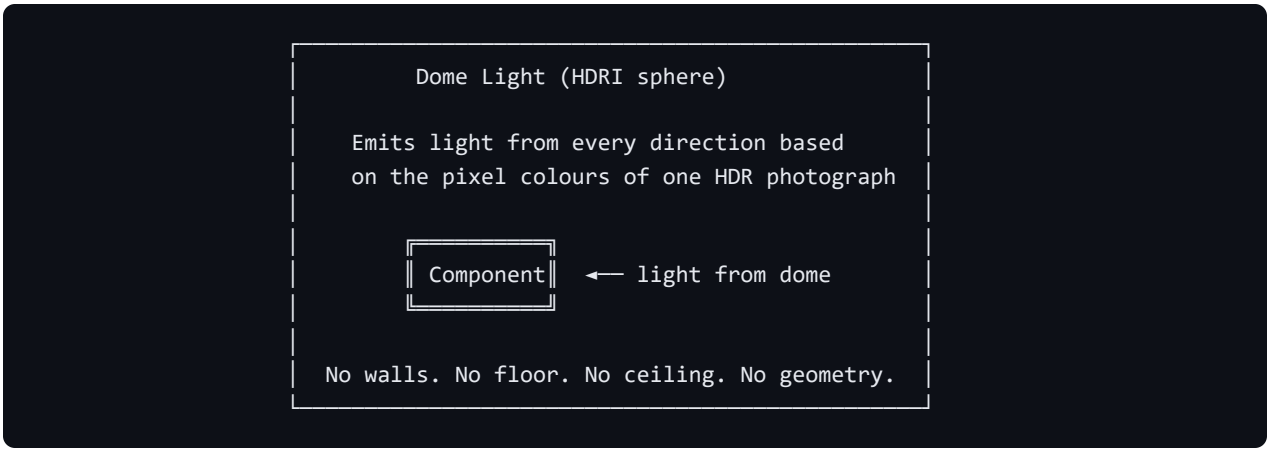
The key insight is that **the room itself is a light transport medium**, not merely a backdrop.

---

3. Dominant Approach in the Literature: Image-Based Lighting (IBL) and Dome Lights

The majority of synthetic data generation pipelines — and many commercial SDG platforms — represent the environment using **Image-Based Lighting (IBL)**: a high-dynamic-range photograph of a real environment (an HDRI map) is mapped onto a virtual hemisphere or sphere that surrounds the scene. This sphere emits light toward the objects based on the radiance values stored in the image.

3.1 What a dome light / HDRI actually does



The light values in an HDRI map were captured by photographing a real environment **once**. At capture time, those bounces did occur — inside the real room where the HDR photograph was taken. But by the time the image is used as a dome light in the simulator, those interactions are **frozen into pixel values**. The simulator does not re-simulate any of the inter-surface bouncing inside your scene; it treats the dome as a fixed external source.

3.2 Fundamental limitations of dome lights

Property	Dome light / HDRI	Physics reality
Scene geometry	None — objects float in a sphere	Walls, floor, ceiling, furniture exist
Inter-object bouncing	Only between the rendered objects themselves	Rays bounce off all room surfaces
Shadow from ceiling	No ceiling exists → no such shadow	Ceiling creates hard shadow boundary directly above
Occlusion by room structure	Light passes through virtual walls	Walls block and redirect light paths
Colour bleeding from surfaces	Absent — wall geometry absent	Yellow wall bleeds warmth onto adjacent component
Light source position	Fixed directional bias from the HDR photo	Fixture positions determine shadow direction
Spatial consistency	Same light field regardless of object position	Light received changes with position in room

The last point is particularly important for detection: in a real room, the lighting on a component changes as it is placed at different positions on a workbench. A dome light provides the same global illumination field regardless of where the object is placed in the scene, which is **physically incorrect** and reduces the appearance diversity of the dataset.

---

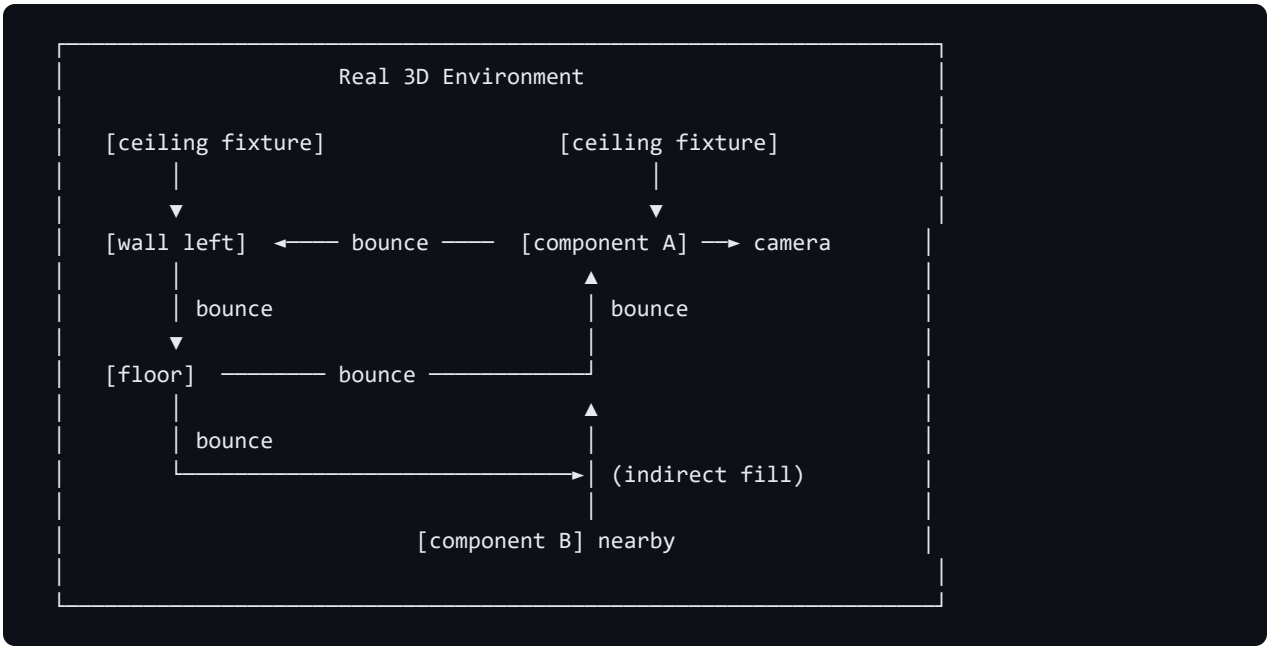
4. The SmartSDG Approach: Full Environment Global Illumination

This pipeline takes a fundamentally different approach. Instead of approximating the environment as an image sphere, the **complete 3D scene geometry** is loaded into the simulator, including:

- Structural elements: walls, floor, ceiling, structural beams.
- Furniture and clutter: workbenches, chairs, shelving, carts.
- Realistic light sources: ceiling-mounted fluorescent fixtures modelled as rectangular area lights at their actual positions.

The path-tracing renderer (NVIDIA Omniverse PathTracing) then solves the **full light transport equation** — the Rendering Equation (Kajiya, 1986) — over this geometry with up to 32 recursive ray bounces per path.

4.1 What this captures that a dome light cannot



The simulated image of Component A contains: 1. **Direct illumination** from the ceiling fixture above. 2. **First-bounce indirect** from the grey concrete floor below the object. 3. **Second-bounce indirect** from the side wall, which may carry colour tint from its surface material. 4. **Inter-object indirect**: light that bounced off Component B arrives at Component A, creating subtle mutual illumination that changes depending on the relative positions of A and B in the scene.

None of items 2, 3, or 4 are reproducible with a dome light, regardless of the quality of the HDRI image used.

4.2 Spatial light variation — the key realism driver

Because the light sources are modelled at specific geometric positions, the illumination of a component **changes as a function of its position on the workbench**. A component placed directly under a ceiling fixture receives strong direct light from above; the same component moved half a metre to the side sits in the indirect zone of the same fixture, receiving more light from the reflective floor below and less from above. This variation — which occurs continuously in real deployments — is captured automatically by the full-environment simulator and **cannot be replicated by any dome light**, because a dome light applies the same spherical radiance field regardless of object position.

4.3 Light source modelling: intensity randomisation

The rectangular area lights in the simulator are randomised per frame:

```
LIGHT_INTENSITY_MAP:
  Low:    [5000,  500]   # mean ± std (arbitrary radiometric units)
  Medium: [20000, 5000]
  High:   [80000, 20000]

LIGHT_TEMPERATURE_MEAN: 8000   # colour temperature (K) — cool-white fluorescent
LIGHT_TEMPERATURE_STD:   500
```

Both intensity and colour temperature are drawn from a normal distribution each frame, producing realistic variation in the apparent brightness and warmth of the illumination across the dataset. Combined with the full path-traced environment, this yields thousands of physically distinct lighting conditions per dataset.

---

5. Comparison with Related Work

Approach	Inter-surface bouncing	Spatial light variation	Ceiling/wall shadow	Colour bleeding	Computational cost
Point / directional light only	✗	✗	✗	✗	Very low
HDRI dome light	Objects only	✗	✗	✗	Low
HDRI dome + proxy geometry	Partial	Partial	Approximate	Partial	Medium
Full 3D environment (this work)	✓	✓	✓	✓	**High**

The additional computational cost is the principal reason that most published SDG pipelines do not use full-environment simulation. This pipeline treats that cost as an investment: by generating a physically realistic dataset once, the downstream detection model requires fewer real labelled images to close the remaining domain gap (quantified via the Active Learning stage).

---

6. Why This Matters for Detection Performance

The visual appearance of an industrial component under realistic multi-bounce illumination differs from its appearance under a dome light in ways that matter directly for the feature-extraction layers of a convolutional or transformer-based detector:

6.1 Edge contrast and shadow sharpness

Real objects in real rooms exhibit partially soft shadows — shadow edges that are not infinitely sharp because the light source has finite area and indirect fill reduces the shadow depth. A dome light produces soft shadows only from the dome itself; it cannot produce the additional fill from a concrete floor. Over-sharp synthetic shadows cause a detector to learn shadow-edge features that do not generalise to real images.

6.2 Surface normal disambiguation

Many industrial components have curved or faceted surfaces. The shading gradient across such a surface — which carries strong information about the surface normal and therefore the object's 3D pose — depends critically on the full spatial distribution of incoming radiance. A dome light that is rotationally uniform produces a lower-contrast shading gradient than the strongly anisotropic illumination of a real room with a few overhead fixtures and reflective walls. A model trained on dome-lit images may fail to disambiguate pose variations that are obvious under real lighting.

6.3 Colour appearance stability

Object colour as captured by a camera is the product of the object's reflectance function and the spectral power distribution of all arriving light paths. Because real rooms have surfaces of varying reflectance (grey walls, warm wooden furniture, reflective metal shelving), the colour of an object shifts slightly depending on where it is in the room. This shift is physically present in full-environment simulation and absent in dome-lit simulation. Training on datasets that contain this shift makes the model more robust to the colour appearance variation seen in real deployments.

---

7. Research Contribution: The Illumination Realism Claim

The primary contribution of this pipeline with respect to illumination is:

> **By replacing image-based dome lighting with full physics-accurate path-traced rendering inside a complete 3D replica of the target deployment environment, the synthetic dataset captures the complete light transport — direct, first-order indirect, and higher-order indirect illumination — that the real camera observes. This closes the photometric component of the sim-to-real gap without requiring any real labelled training images at the generation stage.**

This claim is supported by three specific properties of the implementation:

1. **Complete geometry**: walls, floor, ceiling, and scene clutter are modelled, providing all surfaces needed for multi-bounce indirect light paths.
2. **Physically-based renderer**: NVIDIA Omniverse PathTracing solves the Rendering Equation with configurable bounce depth (default 32), guaranteeing that the simulated radiance values are physically consistent with the real-world measurements they are meant to approximate.
3. **Spatial and spectral randomisation**: light intensity and colour temperature are randomised per frame within calibrated distributions, and the multi-object mode places components at varied positions within the environment, so the dataset contains the full range of position-dependent illumination variation observed in real deployments.

7.1 What this pipeline does that dome-light approaches cannot

Property	Dome light / HDRI	This pipeline
Physics of indirect light	Baked into HDR image at capture time	Re-simulated from first principles per frame
Wall and ceiling interaction	Absent	Fully modelled
Spatial sensitivity	Position-invariant	Illumination changes with object position
Colour bleeding	Absent	Captured from scene surface materials
Generalisation to new positions	Limited	Naturally covered by multi-object randomisation
Reproducibility	HDRI file	AAS passport records exact light parameters

---

8. Camera Sensor Noise: Bridging the Remaining Photometric Gap

Even with physics-accurate ray-traced illumination, one photometric gap remains: real cameras add **sensor noise** during image formation. This includes:

- **Read noise**: thermal and electronic noise present in every pixel, modelled as additive Gaussian noise (σ ≈ 2–15 DN at ISO 100–800).
- **Shot noise**: Poisson-distributed photon-counting noise, dominant at low light.
- **Lens defocus**: slight Gaussian blur from depth-of-field effects when the component is not at the exact focus distance.

None of these are produced by the path-tracer, which models ideal geometric optics. The pipeline therefore includes a **post-processing camera noise module** (`SmartSDG/src/camera_noise.py`) that applies configurable Gaussian pixel noise and optional blur to the generated images after the YOLO conversion step. Labels (bounding box text files) are never modified — only the image pixels are affected.

Configurable parameters (dashboard SDG tab → *Camera noise post-processing*):

Parameter	Range	Effect
`noise_gaussian_sigma`	0 – 30	Sensor read noise strength; 5 = subtle, 15 = strong
`noise_blur_sigma`	0 – 5	Lens defocus radius; 0 = disabled

This makes the noise level an explicit experimental variable: by generating datasets with different σ values and measuring downstream detection performance, the researcher can quantify how much of the remaining sim-to-real gap is attributable to sensor noise rather than geometry or illumination.

---

9. Summary

The table below summarises the photometric realism of this pipeline relative to common alternatives, across the properties that most directly affect a detection model's ability to generalise to real images:

Illumination property	Point lights	HDRI dome	SmartSDG
Direct light from source	✓	✓	✓
First-bounce indirect	✗	Partial	✓
Multi-bounce indirect	✗	✗	✓
Colour bleeding from room surfaces	✗	✗	✓

Illumination property	Point lights	HDRI dome	SmartSDG
Spatial light variation	✗	✗	✓
Ceiling / wall shadow accuracy	✗	✗	✓
Per-frame light randomisation	Possible	Possible	✓
Sensor noise simulation	✗	✗	✓ (post-proc)
Fully reproducible (AAS record)	✗	✗	✓

The combination of full-environment path tracing with per-frame light randomisation and post-processing sensor noise is, to the authors' knowledge, the most complete treatment of the photometric sim-to-real gap available in an open, pipeline-integrated SDG system for industrial object detection.

References

- Kajiya, J. T. (1986). The rendering equation. *ACM SIGGRAPH Computer Graphics*, 20(4), 143–150.
- Müller, T. et al. (2022). Instant neural graphics primitives. *ACM Trans. Graph.*, 41(4).
- Tobin, J. et al. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. *IEEE/RSJ IROS*.
- Tremblay, J. et al. (2018). Training deep networks with synthetic data: Bridging the reality gap by domain randomization. *CVPR Workshops*.
- NVIDIA (2024). *Omniverse Replicator — Path Tracing Renderer*. NVIDIA Developer Documentation.

Active Learning — User Guide

Active Learning (AL) reaches a target accuracy with **far fewer labelled images** by letting the model choose what to label next, instead of labelling everything.

This guide explains every control in the dashboard's 🧠 **Active Learning** and 🎯 **AL Harvest** tabs. It is written for a professional user who is not an AL expert — every dashboard control also has an ⓘ button with a short hint that links here for the full explanation.

The idea in one paragraph

You train a model on a small labelled subset (round 0). The trained model then scores every remaining image in the pool, and a *selection strategy* picks the most useful ones to add to the labelled set. You retrain and repeat for a few rounds. Because the model labels its own "most informative" images first, the accuracy curve usually climbs faster than labelling at random — so you hit your target mAP with a fraction of the labels.

In this pipeline the **pool is your synthetic (SDG) dataset**, and **validation is on real images**, so the gain curve directly measures sim-to-real transfer.

Two ways to run AL

1. On synthetic data (🧠 Active Learning tab)

The selected generated dataset is treated as the unlabelled pool; AL picks the most informative synthetic images each round. Use this to compare strategies and camera geometries on controlled data.

2. On real data via inference (🎯 AL Harvest tab)

Run a trained model over a folder of real, unlabelled images. The most *uncertain* frames are kept with the model's predictions as pre-labels. You review/correct them in the browser (✏️ Edit) and **Accept** them into a `REAL_<name>` dataset — which then becomes trainable / AL-able like any other. This is the human-in-the-loop loop: the model tells you which real frames are worth labelling.

Controls

Dataset = unlabelled pool

The generated dataset whose images form the pool to select from. Generate one in the **Run** tab first. Bigger pools give the strategy more to choose from.

Camera geometry

SDG renders every scene from **two cameras**: - **Replicator** — fixed top-down view - **Replicator_01** — random orbital view

These are different viewpoints and form **separate datasets**. A model trained on one geometry should be compared on that geometry, so pick **one camera** and run it; the run is logged separately (`output/<dataset>__<camera>`) and never mixed with the other. "both" merges them — only use it if you deliberately want a combined model.

Strategy

How the next images are chosen each round: | Strategy | What it does | When to use | |---|---|---| | **random** | uniform random pick | the baseline every method must beat | | **entropy** | most-uncertain images (model least confident) | uncertainty only | | **ppal_box** | uncertainty + diversity by bounding-box geometry | legacy behaviour | | **ppal_feat** | uncertainty + diversity by appearance embeddings | **recommended** |

`ppal_feat` uses the detector's own feature embeddings to avoid selecting many look-alike images — the most faithful to the PPAL paper and usually the best.

Class-balance boost

Real pools are often class-imbalanced. With this on, images containing under-represented classes get a selection bonus, filling in rare classes faster.

Budget: rounds / initial subset / add per round

- **Initial subset** — images labelled and trained on before the first selection (round 0 seed set). Too small and the first model is too weak to score well; a few dozen is typical.
- **AL rounds** — how many select+retrain cycles follow round 0.
- **Add per round** — images added each round (the labelling budget per step).

Constraint: `initial + rounds × add` must not exceed the pool size.

Training: epochs / batch / image size / initial weights

- **Epochs per round** — 3–10 for quick experiments, 50+ for real results.

- **Batch** — larger is faster but uses more GPU memory; on an 8 GB RTX 2080 use 8–16 at 640px with YOLOv12-L, or `-1` for auto-fit.
- **Initial weights** — the pretrained checkpoint round 0 fine-tunes from; **YOLOv12** is the project default.

Reading the Gain Overview

Each line is one AL run (dataset + camera + strategy), plotting **mAP50 vs number of labelled images**. The **gain** column is *best mAP50 − round-0 mAP50*.

To show AL works, run the **same dataset + camera** with `random` and with `ppal_feat`, then compare the two lines at the same number of labelled images — the vertical gap is exactly what Active Learning bought you. Compare the two camera geometries as separate lines to see which viewpoint transfers better.

Where everything is saved

What	Path
AL run outputs + learning curve	`active_learning/output/<dataset>[__<camera>]/al_summary.csv`
Harvest candidates (pending review)	`active_learning/harvest/<name>/{images,labels}`
Accepted real dataset	`SmartSDG/Datasets/yolo_experiments/REAL_<name>/`
AL parameters (record)	the dataset's AAS file under `AAS/datasets/<date>/`

Reproducibility: every AL run writes its parameters into the dataset's AAS file (record mode), so the config and the dataset always travel together.

SDG Hyperparameter Guide

Every knob that affects dataset quality, what it does, and the validated value. **Read this before changing anything** — SDG output is highly sensitive, and a single bad value

silently degrades downstream OD training.

There are three groups, applied at different stages:

```
GROUP 1: scene placement  →  render  →  GROUP 2: quality filters  →  GROUP 3: renderer quality
(prevent problems)                       (discard bad frames)          (image fidelity/speed)
```

Group 1 — Generation-time (how the scene is BUILT)

These prevent problems before rendering. Nothing is deleted by them.

Parameter	Validated	Where	Effect
`--spawn_radius`	**0.2 m**	CLI / `config.yaml: multi_spawn_radius`	Max distance of objects from scene center. Smaller → objects stay inside camera view (no border cuts). Does **not** change object pixel size.
`--min_separation`	**0.05 m**	CLI	Minimum XY distance between object **centers** (collision-aware rejection sampling). Keeps objects from sitting in front of each other → prevents occlusion at the source. `0` disables.
`truck_scale`	0.01	`config.yaml`	World scale of components. **Never change** — 0.1 makes objects 10× too large/close.
`orbit_radius`	1.5	`config.yaml`	Camera 2 distance. Larger → smaller objects on screen

Parameter	Validated	Where	Effect
			(hurts OD). Prefer tuning `spawn_radius` instead.
`focal_length`	48.0	`config.yaml`	Camera lens. Larger → zoomed in (objects bigger, FOV narrower).
`multi_spawn_z_variation`	0.3	`config.yaml`	Vertical spread of spawned objects.
`LIGHT_INTENSITY_MAP`	Low/Medium/High	`config.yaml`	Mean/std lux per `--light_mode`. Keys MUST be `Low` / `Medium` / `High`.

Rule of thumb: if too many frames are being discarded by Group 2, fix it here (smaller spawn radius, larger separation), don't loosen the filters.

Group 2 — Filter-time (what is KEPT after rendering)

Applied during YOLO conversion. **v2 invariant: a filter never removes only a label — if any object fails, the whole frame is discarded** (an image with a visible-but-unlabeled object poisons OD training).

Parameter	Validated	Measures	Frame discarded when...
`--min_visibility`	**0.95**	Replicator `occlusionRatio` — real 3D occlusion **by other objects**	any object is > 5% hidden behind another
`--drop_truncated`	**on**	bbox touching the image border	any object is partially **outside the camera frame**
`--filter_overlaps` + `--iou_threshold`	**on, 0.7**	IoU of final YOLO **label rectangles**	two boxes nearly coincide (IoU ≥ 0.7)

Parameter	Validated	Measures	Frame discarded when…
`--filter_overlaps` + `--containment_threshold`	**on, 0.75**	intersection ÷ **smaller** box area	a small box is ≥ 75% inside a bigger one
(automatic)	always	degenerate zero-area boxes	an object exists but no valid box can be written

min_visibility vs filter_overlaps — not the same thing!

- `min_visibility` = true 3D occlusion (is the object hidden?). The scientifically meaningful one.
- `filter_overlaps` = 2D rectangle geometry. Two boxes can collapse either by **coinciding** (high IoU) or by **nesting** — a small box inside a big one. IoU stays low for the nested case (the union is dominated by the big box), so `containment_threshold` (intersection ÷ smaller-box area) catches it. Depth is ignored: a heavy geometric overlap is dropped regardless of which object is in front, because the cost of mislabelled boxes outweighs keeping the frame.
- With `min_visibility 0.95` active, the overlap filter rarely fires — but the containment guard is what catches a small object swallowed by a big one when its `occlusionRatio` happened to stay under the visibility threshold.

Policies

`--occlusion_policy` / `--truncation_policy`: `frame` (default — discard whole image) or `box` (remove only the label — **avoid**, leaves unlabeled objects).

Group 3 — Renderer / ray-casting quality (`config.yaml: SIMULATION_CONFIG`)

Parameter	Validated	Effect
`renderer`	PathTracing	Physically accurate. `RayTracedLighting` = faster, less realistic.

Parameter	Validated	Effect
`samples_per_pixel_per_frame`	8	Rays per pixel. Main quality/speed knob. 32–64 for final datasets, 8 for iteration (denoiser compensates).
`max_bounces`	32	Light path depth.
`max_specular_transmission_bounces`	6	Bounces through glass/specular.
`max_volume_bounces`	4	Volumetric media.
`denoiser`	True	OptiX AI denoiser — lets low spp look clean.

Advanced carb settings (firefly filter, totalSpp, DLSS mode, RTSubframes…) go in `config.yaml: RTX_CARB_SETTINGS` — applied automatically before the Replicator graph builds. Commented examples are in the config file.

Validated baseline command (2026-06-10)

```
C:\Users\hota02\Documents\omniverse\python.bat main.py ^
  --config "src\config\config.yaml" ^
  --multi_object --components Ylid,YTrailer,Cabin,GlueGun,BLid,BTrailer,FChassis ^
  --light_mode Medium --num_samples 100 --load_env plane ^
  --width 640 --height 480 ^
  --convert_to_yolo --filter_overlaps --drop_truncated
```

(min_visibility 0.95, policies frame, IoU 0.7, spawn 0.2, separation 0.05 are defaults — only the switches need passing.)

How to change a hyperparameter safely

1. Change **one** parameter.
2. Generate a small batch (`--num_samples 20`).
3. Inspect: images visually + the FINAL DATASET SUMMARY (discard rates).
4. Record old → new value and observed effect in `CHANGELOG.md` .
5. Only then scale up to full generation.

Running on Another Machine — Portability & Docker

TL;DR

The pipeline no longer hard-codes any machine path. On a new machine you either:

1. **Set three environment variables**, or
2. **Copy** `pipeline_config.example.json` → `pipeline_config.json` and fill in the paths, or
3. **Do nothing** — auto-detection finds standard install locations.

Check what got resolved at any time:

```
python config_paths.py
```

```
  isaac_python   = ...\omniverse\python.bat            [OK]
  train_python   = ...\envs\yolov12\python.exe         [OK]
  val_real       = ...\val_real                        [OK | MISSING]
```

The three paths:

Key	What	Env var
`isaac_python`	Isaac Sim's `python.bat` (SDG only)	`SMARTSDG_ISAAC_PYTHON`
`train_python`	Python with torch+ultralytics+aas_core3 (OD/AL/CL/inference)	`SMARTSDG_TRAIN_PYTHON`
`val_real`	folder of real validation images (optional)	`SMARTSDG_VAL_REAL`

Resolution order per path: **env var** → `pipeline_config.json` → **auto-detect** → **fallback**. `pipeline_config.json` is git-ignored so each machine keeps its own.

Can Isaac Sim 4.5 run in Docker? — honest answer

Partly, and not on this Windows setup. Here is the real situation so you can decide:

- NVIDIA *does* publish an Isaac Sim **Docker image**, but it is **Linux-only**, ~20 GB, and requires the **NVIDIA Container Toolkit** with GPU passthrough.
- Your current SDG runs on **Windows-native Isaac Sim** (the way NVIDIA ships it for Windows). Windows Isaac Sim does **not** containerize; the Docker image would only run under **WSL2 + GPU passthrough** (Linux inside Windows), which is a large, fragile setup and would not reuse your existing working install.
- So: **Dockerizing the SDG stage on Windows is not worth it.** The native install already works, and the path-config change above already makes it portable to any Windows machine with Isaac Sim installed.

What *is* worth containerizing: the Python stages. OD training, Active Learning, Continual Learning and inference are plain PyTorch/Ultralytics — they containerize cleanly and run on any machine (Linux or Windows+WSL2) with an NVIDIA GPU. A `Dockerfile` for that is provided (see below). The split is:

```
SDG (Isaac Sim)            →  native install (Windows or Linux), path-configured
OD / AL / CL / Inference   →  Docker (portable) OR the same native conda env
```

Docker for the Python stages (recommended for new machines)

`docker/Dockerfile` builds a self-contained image covering **OD Training, Active Learning, Continual Learning, Inference and the Dashboard**. Datasets, weights and configs are bind-mounted at runtime — the image itself stays small and stateless. Isaac Sim is the only stage that stays native.

What you need on the host

Requirement	Windows	Linux
Docker Desktop	✅	Docker Engine
GPU passthrough	Docker Desktop → Settings → Resources → WSL2 GPU	NVIDIA Container Toolkit
NVIDIA driver	525+	525+

One-time build

```
:: Windows
scripts\build_docker.bat

:: Linux / Mac
docker build -t smartsdg -f docker/Dockerfile .
```

First build downloads ~5 GB (PyTorch CUDA base). Subsequent builds use the layer cache and take ~30 s.

Start the dashboard

```
docker compose -f docker/docker-compose.yml up dashboard
```

Then open **http://localhost:8765**. All pipeline stages can be triggered from the dashboard UI with Docker enabled.

Enable Docker mode in the dashboard

In the dashboard ⚙ **Execution** section:

1. Tick **"run training/AL/CL in Docker"** — the toggle is saved to `pipeline_config.json` (`docker_enabled: true`).
2. Training, AL, and CL runs are then automatically wrapped in `docker run --gpus all --shm-size=2g`, with project files mounted at `/workspace` and val-real images at `/valdata`. Path translation from Windows host paths to container paths is handled automatically.

Or set it manually:

```
// pipeline_config.json  (git-ignored, per-machine)
{
  "docker_enabled": true,
  "val_real": "C:/path/to/your/val_real"
}
```

One-off stage commands (without dashboard)

```
# OD Training
docker run --gpus all --shm-size=2g \
  -v "$PWD:/workspace" \
  -e SMARTSDG_WIN_ROOT="$PWD" \
  smartsdg train --aas_file AAS/datasets/<date>/<config>.json

# Active Learning
docker run --gpus all --shm-size=2g -v "$PWD:/workspace" smartsdg al

# Continual Learning
docker run --gpus all --shm-size=2g -v "$PWD:/workspace" smartsdg cl

# Debug shell inside the container
docker compose -f docker/docker-compose.yml run --rm shell
```

Entrypoint modes

Command	What runs
`smartsdg info`	Print torch/CUDA/GPU versions (default)
`smartsdg dashboard`	Start the dashboard on :8765
`smartsdg train [args]`	`OD_training/main.py --mode train [args]`
`smartsdg al [args]`	`AL/main_al.py [args]`
`smartsdg cl [args]`	`CL/main_cl.py [args]`
`smartsdg inference [args]`	`inference/run_inference.py [args]`
`smartsdg shell`	Interactive bash
`smartsdg python script.py`	Run any python script

Known limitations

- **Live webcam inference** does not work in Docker on Windows — USB passthrough is unreliable. Use native inference for the live camera tab.
- **Batch size**: YOLOv12l on 8 GB VRAM needs `batch_size ≤ 8`. The template AAS defaults to 8; adjust in the dashboard form for larger GPUs.
- **Shared memory**: `--shm-size=2g` is required for PyTorch's multi-worker dataloader. The dashboard injects this automatically.

Recommended setup on a fresh machine

With Docker (recommended — no Python install needed)

1. Install **Isaac Sim 4.5** natively (SDG stage only).
2. Install **Docker Desktop** + enable GPU (WSL2 on Windows, nvidia-container-toolkit on Linux).
3. `git clone <repo> && cd SDG_v2`
4. `scripts\build_docker.bat` (Windows) or `docker build -t smartsdg -f docker/Dockerfile .`
5. Create `pipeline_config.json` with `docker_enabled: true` and your `val_real` path.
6. `docker compose -f docker/docker-compose.yml up dashboard` → open http://localhost:8765

Without Docker (conda env)

1. Install **Isaac Sim 4.5** natively.
2. `conda create -n yolov12 python=3.10 && conda activate yolov12`
3. `pip install torch==2.3.0 torchvision==0.18.0 --index-url https://download.pytorch.org/whl/cu121`
4. `pip install -r requirements.txt`
5. `python config_paths.py` — confirm all three paths resolve `[OK]`.
6. `scripts\run_tests.bat` — self-test confirms the whole chain.
7. `scripts\launch_dashboard.bat`.